

A3-4528

21\5\2023

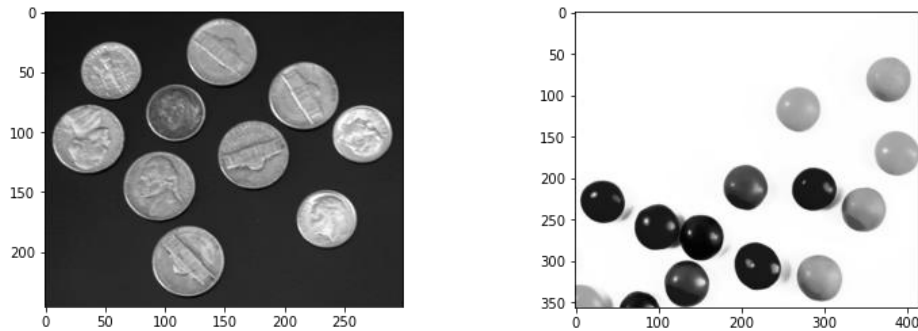
Xinni Song u7439250

Talking with Haoyu Song u7439298

Task1

1. Read the RGB images smarties.jpg and coins.jpg as NumPy arrays and convert RGB images to grayscale (0 mark).

The two grayscale images obtained by using the built-in function are as follows



2. Apply the Canny edge detector on two grayscale images. You may need to try different sets of thresholds for the Canny detector and use the best-performing ones (1 mark).

Hint: You can use OpenCV library functions in this question.

2.1 code detail explanation

- 1) Select the parameter combination of Canny function,
Canny function have two important parameters,

(1). Low Threshold: This threshold is used to determine which pixel values are considered edge candidates. A pixel is considered part of an edge if its gradient value is greater than a low threshold. A lower low threshold results in more edges being detected, but may also contain noise and unimportant edges.

(2.) High Threshold: This threshold is used to determine which edge candidates are considered true edges. A pixel is considered a true edge if its gradient value is greater than a high threshold. A high threshold is mainly used to filter out weak edges and noise, and only keep strong edges.

Choosing an appropriate threshold combination can be adjusted according to specific images and application scenarios. A lower threshold combination will detect more edges, but may also contain more noise and unimportant edges; a higher threshold combination may filter out some weak edges and noise, but may also lose some weaker but still important edge.

For this question, for future circle detection, I will give priority to the canny result with less noise and clearer circle lines on the graph.

Here we tested three parameter combinations,

```
thresholds = [(50, 150), (100, 250), (150, 300)]
```

- 2) Loop and give the canny result of two images.

```
# Iterate over the threshold combinations
```

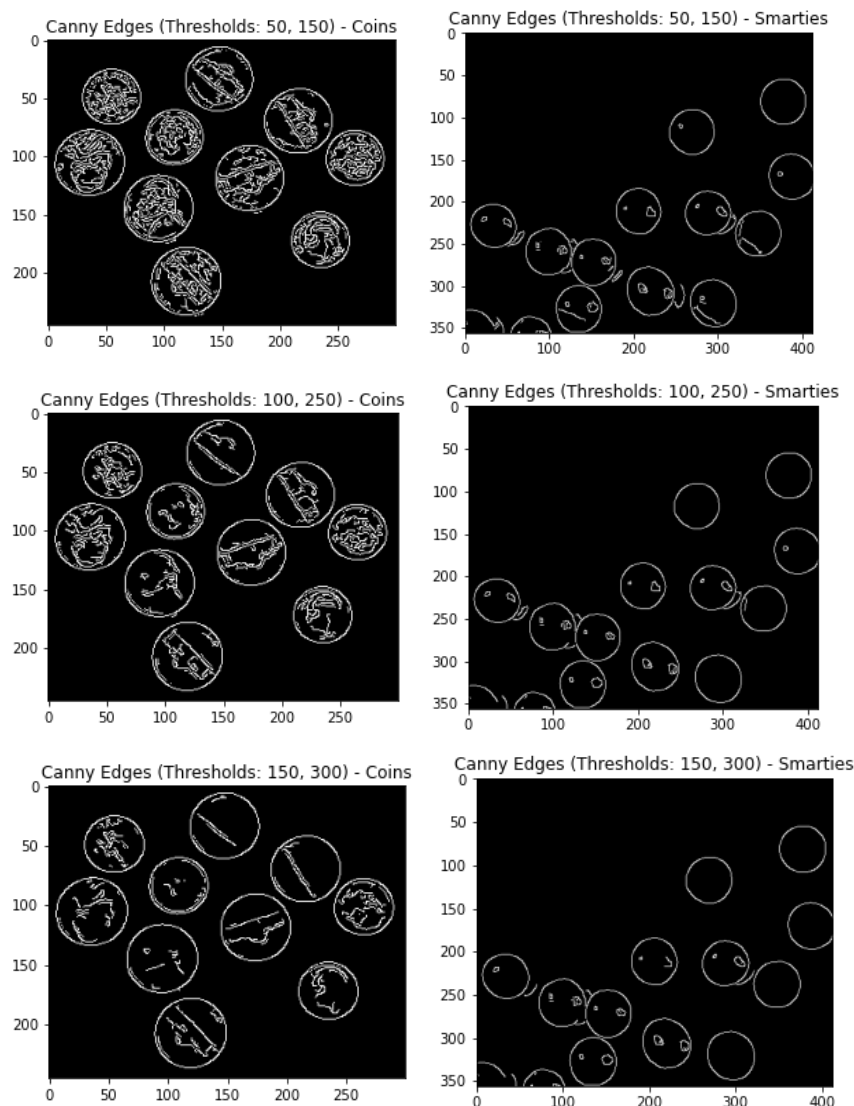
```
for i, (low_threshold, high_threshold) in enumerate(thresholds):  
    edges1 = cv2.Canny(coins_gray, low_threshold, high_threshold)  
    edges2 = cv2.Canny(smarties_gray, low_threshold, high_thresold)  
    ld)
```

3) Display the results.

```
fig, axs = plt.subplots(1, 2, figsize=(10, 5))
axs[0].imshow(edges1, cmap='gray')
axs[0].set_title(f'Canny Edges (Thresholds: {low_threshold}, {high_threshold}) - Coins')
axs[1].imshow(edges2, cmap='gray')
axs[1].set_title(f'Canny Edges (Thresholds: {low_threshold}, {high_threshold}) - Smarties')
plt.show()
```

2.2 Result visualization

The canny results for three combinations of two pictures are as follows,



2.3 Observation

For the results of the coins image, we can observe that as the threshold values increase, the number of detected edge pixels by Canny also decreases. With the threshold combination of (50, 150), the Canny edge detection produces an image where we can clearly see the edges of the person's head on the coin. However, as the

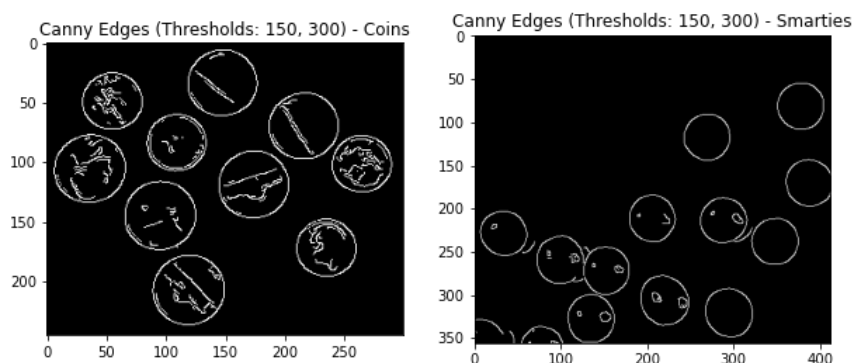
threshold values increase, the clarity of the person's face on the coin gradually decreases. Eventually, with the threshold combination of (150, 300), it becomes difficult to see the person's face on the coin.

For the results of the smarties image, we can see canny results obtained under the three combined thresholds have little change, with only a few different lines.

2.4 Analysis and parameter selecting

As the combination of thresholds increases, both low and high thresholds are increased, which will cause more edges to be filtered out and only the stronger ones to be kept. This leads to the observation above.

Since our purpose is for the subsequent circle detection, some lines that are not circular, such as the face on the coin, may affect the subsequent circle detection, so for coins I choose the canny result from (150, 300). Because in this combination, the face lines on the coin are the least. For smarties, I think there is not much difference between each combination, so I choose (150, 300).



3. Develop a Hough Transformation function to identify circles within images, adhering to the methodology outlined above. Your `hough_circle_transform` function should return a list of circle candidates in the form of (a, b, r) (3 marks).

3.1 Overview of code(contain comments)

The input parameters of the `hough_circle_transform_update` function are as follows:

`edges`: Edge image, a binary image obtained by using the Canny algorithm, where the edge pixel values are non-zero, and used for Hough circle transformation.

`radius_range`: The range of the radius of the circle, which is a tuple containing two elements (`min_radius`, `max_radius`), specifying the range of circle radii to consider.

`radius_step`: The step size of the circle radius, which specifies the interval of the circle radius within a given range.

`num_thetas`: The number of theta values used to parameterize the circle, specifying the number of uniformly distributed theta values in the range of 0 to 2π .

`threshold`: The threshold used to filter circle candidates. Only circles whose corresponding number of votes in the accumulator exceed the threshold will be selected as candidates.

The return value of the `hough_circle_transform_update` function is a list containing candidates for circles detected by the Hough circle transform.

Each circle candidate is represented as a tuple (x, y, radius) , where (x, y) are the coordinates of the center point of the circle and `radius` is the radius of the circle. These candidates represent possible circles detected in the edge image.

```

def hough_circle_transform(edges, radius_range, radius_step
, num_thetas, threshold):
    # Perform edge detection using the Canny algorithm
    # Get the height and width of the edge image
    height, width = edges.shape
    # Create an array of radius values within the specified
range
    radius_range = np.arange(radius_range[0],radius_range[1
],radius_step)
    # Initialize accumulator with zeros
    #a, b, r
    accumulator = np.zeros((height, width, len(radius_range
)))
    # Find the indices of nonzero pixels in the edge image
    y_idx, x_idx = np.nonzero(edges)
    # Iterate through each nonzero pixel and increment the
accumulator
    for r_idx, r in enumerate(radius_range):
        for theta_idx, theta in enumerate(np.linspace(0, 2
* np.pi, num_thetas)):
            # Calculate the center of the circle (a, b) usi
ng the pixel indices (x_idx, y_idx) and radius (r)
            a = np.round(x_idx -
r * np.cos(theta)).astype(int)
            b = np.round(y_idx -
r * np.sin(theta)).astype(int)
            # Filter out invalid indices
            valid_idx = np.where((a >= 0) & (a < width) &
(b >= 0) & (b < height))[0]
            a, b = a[valid_idx], b[valid_idx]
            # Increment the accumulator at the valid indice
s
            accumulator[b, a, r_idx] += 1

    # Find the maximum votes in the accumulator
    max_num = np.max(accumulator)
    # Identify circle candidates with votes above the thres
hold
    candidates = np.argwhere(accumulator / max_num > thresh
old)
    candidates = [(x, y, radius_range[r_idx]) for y, x, r_i
dx in candidates]
    return candidates

```

3.2 Detail explanation

- 1) Initialize some parameters to prepare for subsequent calculations.

Get all radius values within the specified range,

```
height, width = edges.shape
# Create an array of radius values within the specified range
radius_range = np.arange(radius_range[0], radius_range[1], radius_step)
```

Initialize the counter, including three dimensions a, b, r. a, b represent the center of the circle, r represents the radius of the circle.

```
# Initialize accumulator with zeros
#a, b, r
accumulator = np.zeros((height, width, len(radius_range)))
```

Use the np.nonzero function to find the x,y indices of the nonzero pixels in the edge image.

```
# Find the indices of nonzero pixels in the edge image
y_idx, x_idx = np.nonzero(edges)
```

- 2) First, the circle radius range radius_range and theta value range np.linspace(0, 2 * np.pi, num_thetas) are iterated through a nested for loop to calculate the candidate center for each circle.

```
for r_idx, r in enumerate(radius_range):
    for theta_idx, theta in enumerate(np.linspace(0, 2 * np.pi, num_thetas)):
```

- 3) For each center candidate, using the center coordinates (x_idx, y_idx) and the current radius r, and the angle at which it lies, calculate the circle center coordinates (a, b) by, (a, b) is int.

```
a = np.round(x_idx - r * np.cos(theta)).astype(int)
b = np.round(y_idx - r * np.sin(theta)).astype(int)
```

- 4) Filter out the center of the circle within the image range

```
valid_idx = np.where((a >= 0) & (a < width) & (b >= 0) & (b < height))[0]
a, b = a[valid_idx], b[valid_idx]
```

- 5) Ultimately, the value in the accumulator will represent the number of votes for each combination of center location and radius.

```
accumulator[b, a, r_idx] += 1
```

- 6) Find the one with the highest count (a, b, r).

```
# Find the maximum votes in the accumulator
max_num = np.max(accumulator)
```

- 7) Find all (a, b, r) that exceed the set threshold.

```
# Identify circle candidates with votes above the threshold
candidates = np.argwhere(accumulator / max_num > threshold)
candidates = [(x, y, radius_range[r_idx]) for y, x, r_idx in candidates]
```

4. Circle candidates from the previous Hough transform function might include many neighbouring circles. Consequently, the implementation of a non-maximum

suppression algorithm is essential to eliminate these duplicates. Implement non-maximum suppression and incorporate it into the outcomes obtained from the previous question (2 marks).

4.1 Overview of code(contain comments)

circles: A list of candidate circles, each circle is represented by a triplet (a, b, r), where (a, b) is the coordinates of the center of the circle, and r is the radius.

threshold: The threshold used to judge whether two circles overlap. Two circles are considered to overlap if the difference between their center coordinates and radii is less than or equal to this threshold.

The function sorts the candidate circles according to the number of votes from large to small, and then checks one by one whether the circles overlap with other circles in the final circle list, and excludes them if they overlap, and finally returns the circle list after non-maximum suppression.

The non-maximum suppression algorithm can exclude overlapping circles, keep the circle with the highest number of votes, and ensure that no overlapping circles appear in the final circle list, thus providing more accurate circle detection results.

```
def non_max_suppression(circles, threshold):  
    # Sort the circles by their votes (the third value in the tuple)  
    circles = sorted(circles, key=lambda x: x[2], reverse=True)  
  
    # Create a list to store the final circles after non-maximum suppression  
    final_circles = []  
  
    # Loop through each circle  
    for i in range(len(circles)):  
        # Check if the current circle overlaps with any of the final circles  
        overlap = False  
        for j in range(len(final_circles)):  
            # Calculate the absolute differences between the circle parameters  
            a_diff = abs(circles[i][0] - final_circles[j][0])  
            b_diff = abs(circles[i][1] - final_circles[j][1])  
            r_diff = abs(circles[i][2] - final_circles[j][2])  
  
            # If the differences are below the threshold, consider the circles to overlap  
            if a_diff <= threshold and b_diff <= threshold and r_diff <= threshold:  
                overlap = True  
                break
```

```

        # If the circle does not overlap with any of the final circles, add it to the final list
    if not overlap:
        final_circles.append(circles[i])
    return final_circles

```

4.2 Detail explanation

- 1) With this code, we sort the circles from highest to lowest number of votes, with the circle with the most votes at the front of the list. This helps to first select the circle with the most votes as a candidate for the final result when doing non-maximum suppression.

```

circles = sorted(circles, key=lambda x: x[2], reverse=True)

```

- 2) Iterates over the list of circles circles,

```

for i in range(len(circles)):

```

- 3) and for each circle it checks to see if that circle overlaps any of the circles in the final list of circles final_circles. To determine whether two circles overlap, the code calculates the absolute difference between their parameters (center coordinates and radius) and compares it with a given threshold. If the difference between the center coordinates and the radius is less than or equal to the threshold, the two circles are considered to overlap.

```

        for j in range(len(final_circles)):
            # Calculate the absolute differences between the circle parameters
            a_diff = abs(circles[i][0] - final_circles[j][0])
            b_diff = abs(circles[i][1] - final_circles[j][1])
            r_diff = abs(circles[i][2] - final_circles[j][2])

```

- 4) If there is an overlap, the variable overlap is set to True, indicating that the current circle overlaps one of the circles in the final circle list. If there is no overlap, i.e. the variable overlap remains False, the current circle is added to the final circle list.

```

            if a_diff <= threshold and b_diff <= threshold and r_diff <= threshold:
                overlap = True
                break
            if not overlap:
                final_circles.append(circles[i])

```

5. Superimpose circles derived from the previous question on original images and display images with circles (1 mark).

Hint: You may need to use the circle function from OpenCV.

5.1 Code implement

- 1) For the `hough_circle_transform` of coins and smarties pictures, the parameters I set are as follows,

Parameter name	radius_range	radius_step	num_thetas	threshold
value	[10,30]	1	100	0.5

```

circles = hough_circle_transform(coins_edge, radius_range,
                                radius_step, num_thetas, threshold)
circles_smarties = hough_circle_transform(smarties_edge, radius_range,
                                          radius_step, num_thetas, threshold)

```

- 2) For the `non_max_suppression` I used thresh = 50

```

final_circles = non_max_suppression(circles, 50)
final_circles_smarties = non_max_suppression(circles_smarties, 50)

```

- 3) draw circle on graph

```

for x, y, r in final_circles:
    cv2.circle(coins_bgr, (x, y), r, (255, 0, 0), 2)
for x, y, r in final_circles_smarties:
    cv2.circle(smarties_bgr, (x, y), r, (255, 0, 0), 2)

```

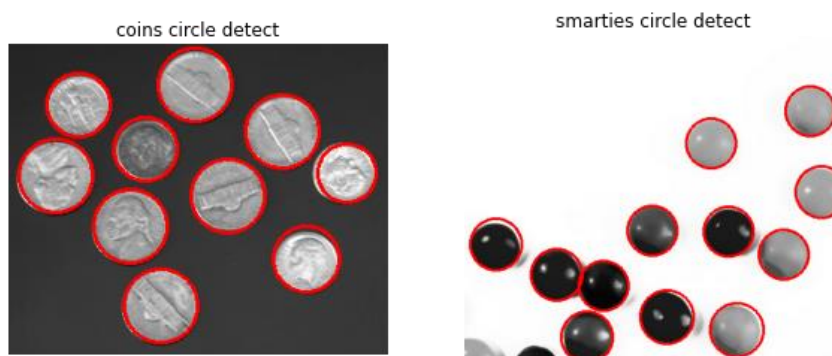
- 4) display the result.

```

plt.subplot(1, 2, 1), plt.imshow(coins_bgr)
plt.title('coins circle detect'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(smarties_bgr)
plt.title('smarties circle detect'), plt.axis('off')

```

5.2 Result visualization



5.3 Observation

The red circle represents the result of Hough detection. It can be seen that the circular outer edges of all coins images have been successfully marked, and there will be a little displacement on individual coins.

The smarties image is also like this, there will be a little offset on some smarties.

5.4 Analysis

The offset may be due to factors such as noise in the image, imperfect edge detection, and parameter selection of the Hough circle detection algorithm. Noise and image quality problems may lead to inaccurate edge detection, which affects the results of Hough circle detection. In addition, the parameter settings of the Hough circle detection algorithm (such as radius range, step size, threshold, etc.) will also affect the detection results.

In order to improve the results, later I will try to tune the parameters for more accurate edge detection and circle detection.

6. Analyse images derived from question 5. Assess the algorithm's success in detecting all circles and the presence of any false positives or false negatives. Provide insight into the effectiveness or shortcomings of the results (1.5 marks).

Based on the analysis of the image in question 5, we can evaluate the performance of the algorithm in detecting all the circles and false positives or missed detections.

Based on the analysis of the image in question 5, we can evaluate the performance of the algorithm in detecting all the circles and false positives or missed detections.

In the coin image, the algorithm successfully detected all the circles and successfully marked their outer edges. This shows that the algorithm achieved a good success rate in coin detection, accurately finding the circular object in the image.

However, we also observed slight offsets for individual coins, which could be due to noise in the image or imperfect edge detection.(image as follow)



This caused some inaccuracies, making the edge markers of the circle slightly off from their actual positions. This can be seen as a kind of shortcoming of the algorithm, and the accuracy of edge detection and circle detection may need to be further improved.

In Smarties images, we also observe a similar situation. The algorithm successfully detected the rounded edges of most smart candies, but there were some offsets of individual smart candies. This can also be attributed to image noise and limitations of edge detection. In addition, there are some candies that belong to the edge, and the complete candies cannot be displayed on the original image, so my function failed to detect the incomplete candies on the edge. (As follow)



In summary, the algorithm performs well in detecting mostly circular objects, but there are still some errors and offsets. In order to improve the accuracy of the algorithm, you can consider adjusting the parameters of edge detection and circle detection, as well as applying image enhancement and denoising techniques. In addition, you can try other circle detection algorithms or use machine learning methods to further improve the precision and accuracy of detection.

7. Examine the impact of different parameters of Circle Hough Transform (e.g., radius range, radius increment, theta count and a variety of thresholds) (1.5 marks).

7.1 Experiment method

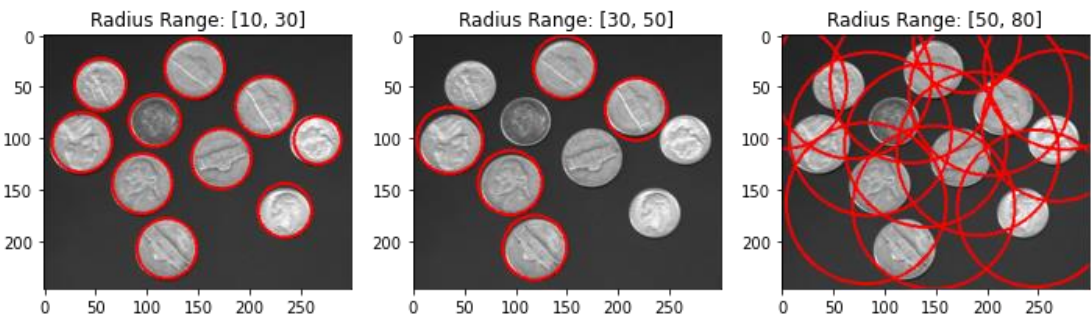
To evaluate the effect of different parameters of the circular Hough transform on the results, I will use the method of control variables. Only one parameter is changed in each experiment, and other parameters are kept unchanged (other parameters use the values of the previous question) to observe its impact on the results. Here use the coins image to complete the following process.

Here is my experimental design:

Radius range: Keep other parameters unchanged, try different radius ranges,

radius_range_list	[10,30]	[30,50]	[50,80]
-------------------	---------	---------	---------

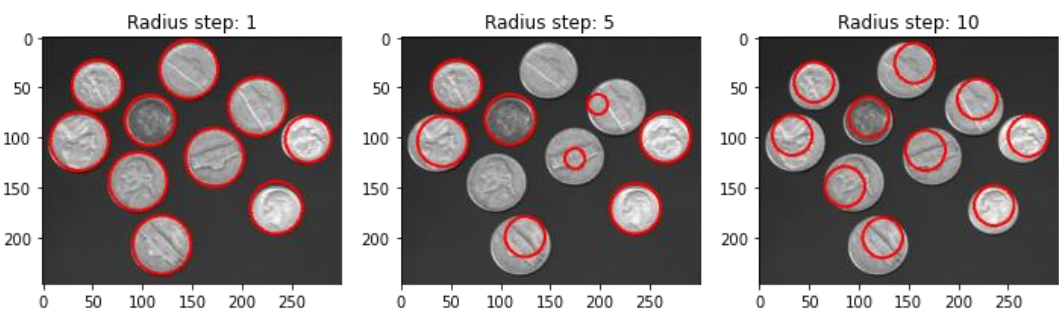
The result is as follow,



radius increment: Keep other parameters unchanged, try different radius increment values,

radius_step	1	5	10
-------------	---	---	----

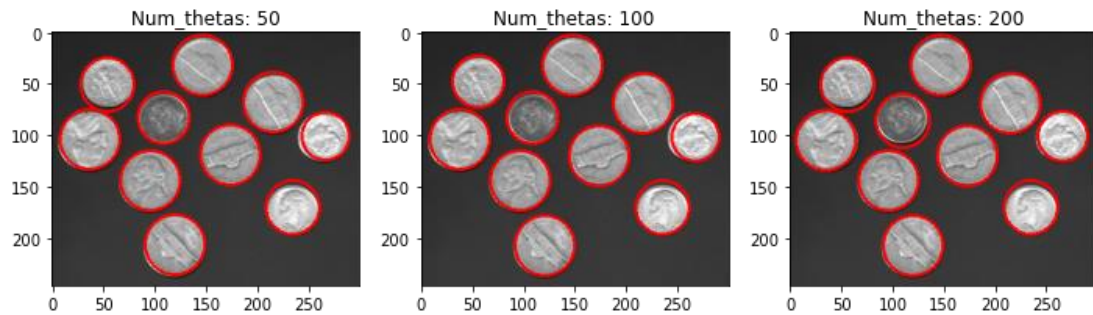
The result is as follow,



theta count: Keep other parameters unchanged, try different Theta numbers,

num_thetas_list	50	100	200
-----------------	----	-----	-----

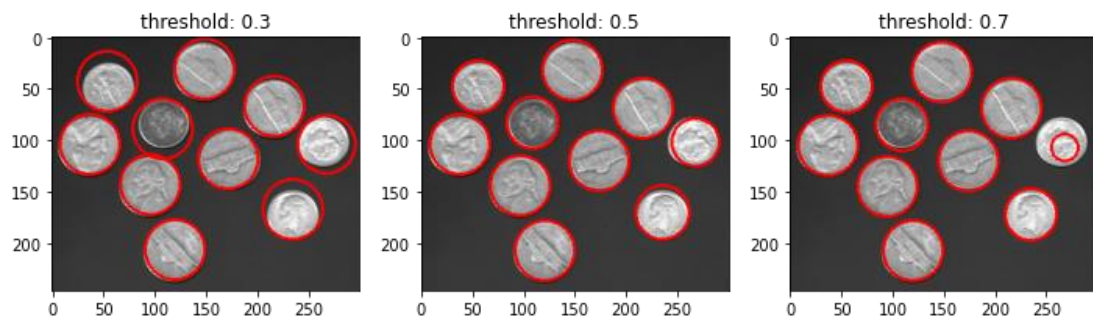
The result as follow,



threshold: Keep other parameters unchanged, try different thresholds,

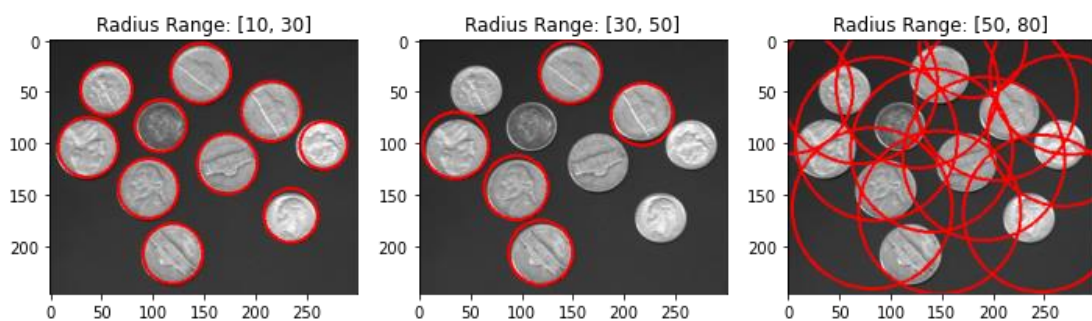
threshold_list	0.3	0.5	0.7
----------------	-----	-----	-----

The result as follow,

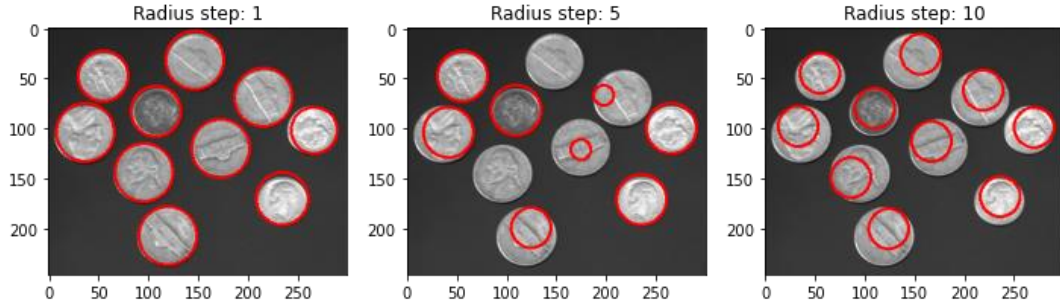


7.2 Observation and analysis

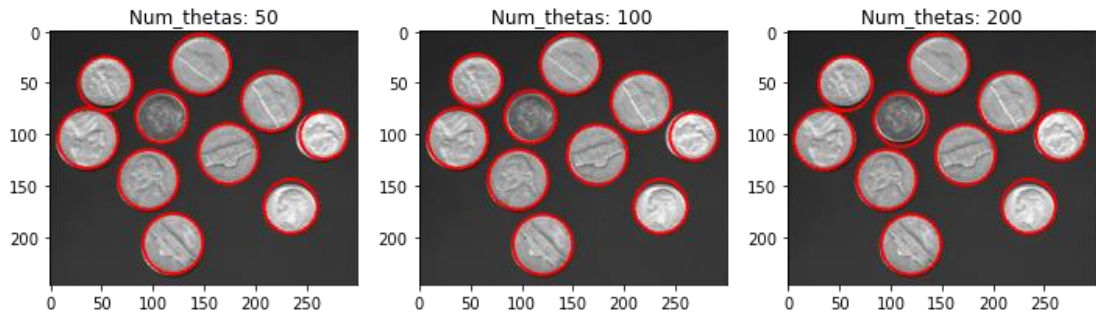
Radius range: Using a smaller radius range (10, 30), we observe that all coins in the image are successfully detected. This result may be because most of the coins in the original image have a radius range between [10, 30]. And when we use a larger radius range, such as [30, 50], we only successfully detected five coins, because this range exceeds the radius of the coin in the original image, resulting in inaccurate detection results. When the radius range is [50, 80], the result is very confusing, and no coins are successfully detected, which shows that the circle in the original image does not have a radius between [50, 80].



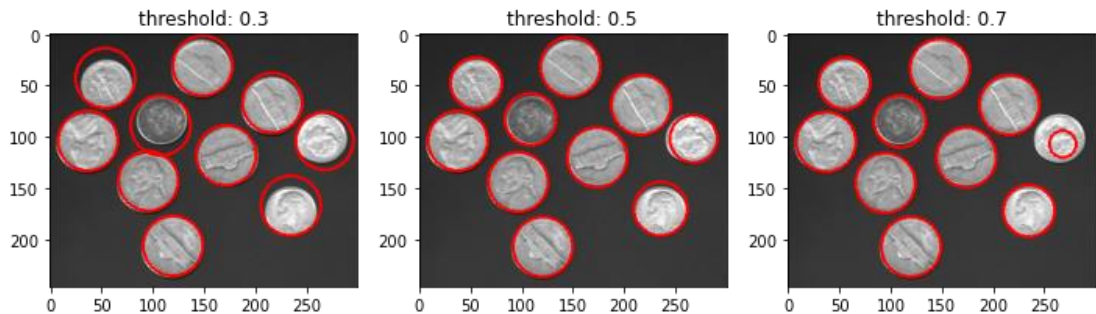
Radius step: When using a smaller radius increment value of "1", we obtained the best detection results, correctly detecting all the coins. However, with a larger radius increment value of "5", the number of well-detected coins decreased to only four, and the detected circles were smaller than the actual coins. Using a radius increment of "10" did not accurately detect the size of the coin circles, but it did manage to detect the correct number of coins.



num_thetas: From the results, we can observe that there is almost no difference in the detection results when num_thetas is "50", "100", and "200". The main difference may be that smaller num_thetas takes less time to compute.



threshold: A smaller threshold of 0.3 can detect more circles, but it may also cause some false detections, such as the detection results of coins are too large. A larger threshold of 0.7 can filter out some weaker circular edges, but also causes some missed detections. When the threshold is set to 0.5, the detection result is the best.



7.3 Conclusion

The selection of the radius range has a great influence on the detection results. Therefore, when selecting the radius range, an appropriate radius should be selected according to the radius of the circle in the original image. The choice of radius increment value also has a certain influence on the detection results. A smaller increment value can provide finer detection, but increases the computational complexity; a larger increment value may lead to missed detection or increase the error of detection results. Observations show that there is almost no noticeable difference in detection results when num_thetas is set to a small value (such as 50) and a large value (such as 200). Smaller num_thetas can reduce computation time, but may result in loss of detail. The choice of threshold affects the number and accuracy of detected circles. Smaller thresholds can detect more circles, but may introduce some false detections; larger thresholds can filter out weaker circular edges, but may also lead to missed detections. According to the

observation results, choosing an appropriate threshold (such as 0.5) can get better detection results.

The radius range has a great influence on the detection results, and an appropriate range should be selected according to the radius of the circle in the original image. The choice of radius increment also affects the results, smaller increments provide finer detection but increase computational complexity, larger increments may lead to missed detections or increased errors. The setting of num_thetas have little difference in results between small and large values. Smaller values can reduce calculation time but may lose details. The choice of threshold affects the number and accuracy of detected circles. Smaller thresholds can detect more circles but may introduce false detections. Larger thresholds can filter weak edges but may miss detection. Appropriate choice of threshold (such as 0.5) can get better results.

Generally speaking, we should combine the observation of the original image to choose the appropriate parameters.

8. An allocation of two marks will be awarded for time and memory efficiency (for both algorithmic asymptotic analysis and actual program runtime aspects) demonstrated by the functions in all questions (2 marks).

8.1 For hough_circle_transform

`hough_circle_transform(edges, radius_range, radius_step, num_thetas, threshold)`

● Time Asymptotic analysis

1. Get the height and width of the edge image, which takes $O(1)$ time.
2. Create an array of radius values within a radius, with a step size of radius_step. According to the given radius range, the length of this array is $O(1)$.
3. Initialize an accumulator matrix with dimensions (height, width, len(radius_range)) and initialize all its elements to zero. The time complexity of this step is $O(\text{height} * \text{width} * \text{len}(\text{radius_range}))$.
4. Find the indices of the non-zero pixels in the edge image, i.e. the coordinates of the edge pixels. This step takes $O(n)$ time.
5. For each combination of radius and angle values, iterate through the accumulator. The number of iterations of this loop is $\text{len}(\text{radius_range}) * \text{num_thetas}$, where $\text{len}(\text{radius_range})$ is the length of the array of radius values and num_thetas is the number of angle values.
6. In each iteration, calculate the center coordinates (a, b) of the circle by using the pixel index and the radius value. This calculation involves some trigonometric operations and rounding operations, and its time complexity is $O(1)$.
7. Filter out invalid indices, i.e. indices that go beyond the boundaries of the image. The time complexity of this filtering operation is $O(1)$.
8. At a valid index, increment the value at the corresponding location in the accumulator by 1. The time complexity of this operation is $O(1)$.
9. Find the maximum number of votes in the accumulator. This operation needs to traverse the entire accumulator matrix, and the time complexity is $O(\text{height} * \text{width} * \text{len}(\text{radius_range}))$.
10. Based on the threshold, identify circle candidates with votes above the threshold. The time complexity of this operation is $O(\text{height} * \text{width} * \text{len}(\text{radius_range}))$.

In summary, the total time complexity of the algorithmic asymptotic analysis of this function is $O(\text{height} * \text{width} * \text{len}(\text{radius_range}) * \text{num_thetas})$.

- **actual program runtime**

I use the time module to record the start time and end time of the function, and the calculated total running time of the function is 0.7530 seconds.

- **Memory Asymptotic analysis**

1. The space occupied by the edge image is $O(\text{height} * \text{width})$, where height is the height of the image and width is the width of the image.
2. The space occupied by the radius range array is $O(\text{len}(\text{radius_range}))$, where $\text{len}(\text{radius_range})$ is the number of radii in the radius range.
3. The space occupied by the accumulator matrix is $O(\text{height} * \text{width} * \text{len}(\text{radius_range}))$, where height is the height of the image, width is the width of the image, and $\text{len}(\text{radius_range})$ is the number of radii within the radius range.
4. The space occupied by an array of non-zero pixel indices is $O(n)$, where n is the number of non-zero pixels in the edge image.
5. The space usage of other local variables and temporary arrays is negligible because their size does not vary with the input size.

In summary, the space complexity of this function is $O(\text{height} * \text{width} * \text{len}(\text{radius_range}) + n)$, where height and width are the height and width of the image, $\text{len}(\text{radius_range})$ is the number of radii within the radius, and n is the number of non-zero pixels in the edge image.

8.2 For non_max_suppression:

[`non\_max\_suppression\(circles, 50\)`](#)

- **Time Asymptotic analysis**

1. The time complexity of sorting the circles by their number of votes is $O(n \log n)$, where n is the number of circles.
2. The time complexity of non-maximum suppression for each circle is $O(n^2)$, where n is the number of circles.
3. The total time complexity is $O(n^2 + n \log n)$, so it can be simplified to $O(n^2)$, where n is the number of circles.

In summary, the time complexity of this function is $O(n^2)$, where n is the number of circles.

- **actual program runtime**

I use the time module to record the start time and end time of the function, and the calculated total running time of the function is 0.0030 seconds.

- **Memory Asymptotic analysis**

1. The space occupied by the circle list is $O(n)$, where n is the number of circles.
2. The space occupied by the final circle list (`final_circles`) is also $O(n)$, because in the worst case, no circles are removed.
3. The space usage of other local variables and temporary variables is negligible because their size does not vary with the input size.

In summary, the space complexity of this function is $O(n)$, where n is the number of circles.

Task 2

1. Read the RGB images moutain1.jpg and mountain2.jpg as NumPy arrays (0 mark)



The left one is mountain1, the right one is mountain2.

2. Implement the homography function with the interface described above, which takes normalised coordinates as inputs (2 marks).

2.1 Overview of homography function:

```
def homography(u2_trans, v2_trans, u_base, v_base):
    A = []
    dim = len(u_base)
    for i in range(dim):
        A.append([-u_base[i], -v_base[i], -
1, 0, 0, 0, u_base[i]*u2_trans[i], v_base[i]*u2_trans[i], u
2_trans[i]])
        A.append([0, 0, 0, -u_base[i], -v_base[i], -
1, u_base[i]*v2_trans[i], v_base[i]*v2_trans[i], v2_trans[i
]])
    A = np.asarray(A)
    U, D, V = np.linalg.svd(A)
    h_mat = V[-1, :].reshape(3, 3)
    h_mat /= h_mat[2, 2]
    return h_mat
```

2.2 Detail explanation:

- 1) Initialize A empty list and get the number of input points

```
A = []
dim = len(u_base)
```

- 2) For each correspondence $x_i \leftrightarrow X_i$ compute A ,

$x_i = PX_i$ represents the two-dimensional point x_i obtained by projecting the three-dimensional point P_i through the projection matrix P .

The equation $x_i \times PX_i = 0$ is one of the constraints, x_i is parallel to PX_i , so the cross product result is 0.

$$x_i = PX_i \Rightarrow x_i \times PX_i = 0$$

Because

The cross product of two vectors **a** and **b** (**a**×**b**) can be written as a product of an anti-symmetric matrix **[a]_x** and the vector **b**

$$a \times b = [a]_x b = -[b]_x a$$

The appropriate anti-symmetric matrix **[a]_x** is:

$$[a]_x = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}$$

From the above formula we can deduce that

$$\left. \begin{aligned} \mathbf{x}_i &= (x_i, y_i, w_i)^T \\ \mathbf{P}\mathbf{X}_i &= \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \mathbf{p}_3^T \end{bmatrix} \mathbf{X}_i = \begin{pmatrix} \mathbf{p}_1^T \mathbf{X}_i \\ \mathbf{p}_2^T \mathbf{X}_i \\ \mathbf{p}_3^T \mathbf{X}_i \end{pmatrix} \end{aligned} \right\} \mathbf{x}_i \times \mathbf{P}\mathbf{X}_i = \begin{pmatrix} y_i \mathbf{p}_3^T \mathbf{X}_i - w_i \mathbf{p}_2^T \mathbf{X}_i \\ w_i \mathbf{p}_1^T \mathbf{X}_i - x_i \mathbf{p}_3^T \mathbf{X}_i \\ x_i \mathbf{p}_2^T \mathbf{X}_i - y_i \mathbf{p}_1^T \mathbf{X}_i \end{pmatrix}$$

Then we get the form as follow,

$$\begin{bmatrix} 0^T & -w_i \mathbf{X}_i^T & y_i \mathbf{X}_i^T \\ w_i \mathbf{X}_i^T & 0^T & -x_i \mathbf{X}_i^T \\ -y_i \mathbf{X}_i^T & x_i \mathbf{X}_i^T & 0^T \end{bmatrix} \begin{pmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \end{pmatrix} = \mathbf{A}_i \mathbf{p} = 0$$

Therefore, in order to find the matrix P, which is the h_mat in the function, I need to construct the A matrix first, and the construction of the A matrix is as follows

```
for i in range(dim):
    A.append([-u_base[i], -v_base[i], -
1, 0, 0, 0, u_base[i]*u2_trans[i], v_base[i]*u2_trans[i], u
2_trans[i]])
    A.append([0, 0, 0, -u_base[i], -v_base[i], -
1, u_base[i]*v2_trans[i], v_base[i]*v2_trans[i], v2_trans[i]
]])
A = np.asarray(A)
```

- 3) Compute the SVD of A. The solution for p is the last row of V

```
# Compute the singular value decomposition of A
U, D, V = np.linalg.svd(A)
# Extract the homography matrix from the last row of V
h_mat = V[-1, :].reshape(3, 3)
```

- 4) Normalize the h matrix

```
h_mat /= h_mat[2, 2]
```

3. Implement a function to generate normalisation transformation matrices (refer to lecture slides) for point coordinates in the homogeneous coordinate, integrate this normalisation step to the homography estimation function above and put them into a function named `homography_w_normalisation` (2 marks).

3.1 Code Overview

```
def normalize_coordinates(points, img):  
    h, w = img.shape[:2]  
    T_norm = np.array([[w + h, 0, w / 2],  
                       [0, w + h, h / 2],  
                       [0, 0, 1]])  
  
    # Normalize the points  
    normalized_points = np.dot(T_norm, points.T).T  
    return normalized_points, T_norm  
  
def homography_w_normalisation(u2_trans, v2_trans, u_base,  
                               v_base, img_base, img_trans):  
    # Combine the u and v coordinates into homogeneous coordinates  
    points_trans = np.column_stack((u2_trans, v2_trans, np.ones_like(u2_trans)))  
    points_base = np.column_stack((u_base, v_base, np.ones_like(u_base)))  
  
    # Normalize the coordinates of the points  
    normalized_points_trans, T_trans = normalize_coordinates(points_trans, img_trans)  
    normalized_points_base, T_base = normalize_coordinates(points_base, img_base)  
  
    # Estimate the homography matrix using the normalized points  
    h_mat = homography(normalized_points_trans[:, 0], normalized_points_trans[:, 1],  
                       normalized_points_base[:, 0], normalized_points_base[:, 1])  
  
    # Denormalize the homography matrix  
    H = np.linalg.inv(T_trans).dot(h_mat).dot(T_base)  
    H /= H[2, 2]  
    return H
```

Compared with the parameters in the T2.2, `img_base`, `img_trans` are added here, which are image arrays.

3.2 Detail explanation

First of all, the principle of this question comes from the content of PPT, as shown in the figure below,

Normalised DLT algorithm

Objective
Given $n \geq 4$ 2D to 2D point correspondences $\{x_i \leftrightarrow x'_i\}$, determine the 2D homography matrix H such that $x'_i = Hx_i$
Algorithm
(i) Normalize points $\tilde{x}_i = T_{\text{norm}} x_i, \tilde{x}'_i = T'_{\text{norm}} x'_i$
(ii) Apply DLT algorithm to $\tilde{x}_i \leftrightarrow \tilde{x}'_i$,
(iii) Denormalize solution $H = T_{\text{norm}}^{-1} \tilde{H} T'_{\text{norm}}$

$$T_{\text{norm}} = \begin{bmatrix} w+h & 0 & w/2 \\ 0 & w+h & h/2 \\ 0 & 0 & 1 \end{bmatrix}^{-1} \quad T'_{\text{norm}} = \begin{bmatrix} w'+h' & 0 & w'/2 \\ 0 & w'+h' & h'/2 \\ 0 & 0 & 1 \end{bmatrix}^{-1}$$

Figure principle of normalized DLT

3.2.1 homography_w_normalisation

- 1) The function `homography_w_normalisation` takes as input the transformed image coordinates (`u2_trans` and `v2_trans`), the base image coordinates (`u_base` and `v_base`), as well as the transformed and base images (`img_trans` and `img_base`).

```
def homography_w_normalisation(u2_trans, v2_trans, u_base, v_base,
                               img_base, img_trans):
```

- 2) Combine `u2_trans` and `v2_trans`, `u2_base` and `v2_base` coordinates into homogeneous coordinates by stacking them horizontally with an array using `np.column_stack`.

```
    points_trans = np.column_stack((u2_trans, v2_trans, np.ones_like(u2_trans)))
    points_base = np.column_stack((u_base, v_base, np.ones_like(u_base)))
```

- 3) The `normalize_coordinates` function is called twice to normalize the coordinates of the points in the transformed image and the base image. It returns normalized points and transformation matrix. It will be explained in detail in `normalize_coordinates` later

```
    normalized_points_trans, T_trans = normalize_coordinates(points_trans, img_trans)
    normalized_points_base, T_base = normalize_coordinates(points_base, img_base)
```

- 4) Call the `homography` function in question 2 and use the normalized coordinates as input to get the homography matrix.

```
    h_mat = homography(normalized_points_trans[:, 0], normalized_points_trans[:, 1],
                       normalized_points_base[:, 0], normalized_points_base[:, 1])
```

- 5) The resulting homography matrix, `h_mat`, is denormalized by multiplying it with the inverse of the transformation matrix of the transformed image, `T_trans`, followed by the transformation matrix of the base image, `T_base`.

(iii) Denormalize solution $H = T_{\text{norm}}^{-1} \tilde{H} T_{\text{norm}}$

```
H = np.linalg.inv(T_trans).dot(h_mat).dot(T_base)
```

- 6) The denormalized homography matrix, H, is then divided by its last element (H[2, 2]) to normalize it further.

```
H /= H[2, 2]
```

3.2.2 normalize_coordinates

- 1) The function `normalize_coordinates` takes as input the points (homogeneous coordinates) and the `img` (the image).

```
def normalize_coordinates(points, img):
```

- 2) The line `h, w = img.shape[:2]` extracts the height and width of the image. `img.shape` returns a tuple with the dimensions of the image, and `[:2]` extracts the first two elements of the tuple, which represent the height and width.

```
h, w = img.shape[:2]
```

- 3) The normalization matrix `T_norm` is created using `np.array`. It is a 3x3 matrix that will be used to normalize the points. The normalization matrix `T_norm` is defined as:

$$T_{\text{norm}} = \begin{bmatrix} w+h & 0 & w/2 \\ 0 & w+h & h/2 \\ 0 & 0 & 1 \end{bmatrix}^{-1} \quad T'_{\text{norm}} = \begin{bmatrix} w'+h' & 0 & w'/2 \\ 0 & w'+h' & h'/2 \\ 0 & 0 & 1 \end{bmatrix}^{-1}$$

```
T_norm = np.array([[w + h, 0, w / 2],  
                  [0, w + h, h / 2],  
                  [0, 0, 1]])
```

- 4) The line `normalized_points = np.dot(T_norm, points.T).T` applies the normalization matrix `T_norm` to the points. The points are transposed (`points.T`) and multiplied with `T_norm` using `np.dot`. The resulting points are then transposed back to their original shape.

```
normalized_points = np.dot(T_norm, points.T).T
```

- 5) The normalized points and the normalization matrix are returned as the output of the function.

```
return normalized_points, T_norm
```

4. Use SIFT feature descriptor to find out matching feature points from both images. Sample half of these corresponding points ($\lceil N/2 \rceil$) using random seed 42, convert the coordinates into NumPy arrays and store them as npy files (1 mark).

4.1 Code overview

```
def find_matching_points(image1, image2):  
    # Create a SIFT object  
    sift = cv2.SIFT_create()  
    # Detect keypoints and compute descriptors for image1  
    keypoints1, descriptors1 = sift.detectAndCompute(image1  
, None)  
    # Detect keypoints and compute descriptors for image2  
    keypoints2, descriptors2 = sift.detectAndCompute(image2  
, None)  
    # Create a BFMatcher object with L2 distance and cross-  
    checking  
    matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)  
    # Match descriptors of image1 and image2 keypoints  
    matches = matcher.match(descriptors1, descriptors2)  
    # Randomly select half of the matches  
    np.random.seed(42)  
    num_matches = len(matches)  
    num_samples = int(np.ceil(num_matches / 2))  
    random_indices = np.random.choice(range(num_matches), s  
ize=num_samples, replace=False)  
    selected_matches = [matches[i] for i in random_indices]  
    # Extract the selected keypoints' coordinates from imag  
e1 and image2  
    selected_points1 = np.array([keypoints1[match.queryIdx]  
.pt for match in selected_matches])  
    selected_points2 = np.array([keypoints2[match.trainIdx]  
.pt for match in selected_matches])  
    # Return the selected points from image1 and image2  
    return selected_points1, selected_points2  
  
# get the match points  
selected_points1, selected_points2 = find_matching_points(m  
ountain1, mountain2)  
  
# save as npy  
np.save('selected_points1.npy', selected_points1)  
np.save('selected_points2.npy', selected_points2)
```

4.2 Detail explanation

- 1) The function `find_matching_points` takes as input `image1` and `image2`, two images for which matching points need to be found.

```
def find_matching_points(image1, image2):
```

- 2) Create a SIFT (Scale-Invariant Feature Transform) object. SIFT is a feature extraction algorithm, which is widely used in the detection and description of image feature points in the field of computer vision.

```
sift = cv2.SIFT_create()
```

- 3) Keypoints1 and descriptors1 are computed by applying the SIFT algorithm on `image1` using the `detectAndCompute` method. `keypoints1` is a list of keypoint objects, each keypoint object contains the coordinates and other related information of the keypoint. `descriptors1` is a matrix of descriptors, where each row corresponds to a feature description for a keypoint. Similarly, `keypoints2` and `descriptors2` are computed for `image2`.

```
keypoints1, descriptors1 = sift.detectAndCompute(image1, None)  
keypoints2, descriptors2 = sift.detectAndCompute(image2, None)
```

- 4) The matcher object is created using `cv2.BFMatcher` with the L2 distance metric and cross-checking enabled. It represents a brute-force matcher that matches descriptors based on their Euclidean distances.

```
matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)
```

- 5) The `match` method is called on the matcher object, passing the descriptors of `image1` and `image2`. It returns a list of matches between the keypoints of the two images.

```
matches = matcher.match(descriptors1, descriptors2)
```

- 6) The random seed is set to ensure reproducibility.

```
np.random.seed(42)
```

- 7) The number of matches, `num_matches`, is determined. Then, `num_samples` is calculated as half of `num_matches` (rounded up).

```
num_matches = len(matches)  
num_samples = int(np.ceil(num_matches / 2))
```

- 8) Random indices are generated without replacement using `np.random.choice`, and the corresponding matches are extracted into `selected_matches`.

```
random_indices = np.random.choice(range(num_matches), size=num_samples, replace=False)  
selected_matches = [matches[i] for i in random_indices]
```

- 9) Extract the selected keypoints' coordinates from `image1` and `image2`. For each selected match, the query keypoint's coordinates from `image1` are accessed using `match.queryIdx`, and the train keypoint's coordinates from `image2` are accessed using `match.trainIdx`. The coordinates are stored in `selected_points1` and `selected_points2` as arrays. And return them.

```
selected_points1 = np.array([keypoints1[match.queryIdx].pt for match in selected_matches])
```

```

        selected_points2 = np.array([keypoints2[match.trainIdx].pt for match in selected_matches])

```

- 10) Call the function to get the matching points of the two pictures and save them in the npy file.

```

selected_points1, selected_points2 = find_matching_points(mountain1, mountain2)
# save as npy
np.save('selected_points1.npy', selected_points1)
np.save('selected_points2.npy', selected_points2)

```

5. The correspondences obtained from SIFT feature might contain errors (outliers). Thus, the RANSAC algorithm should be used to fit a robust homography estimation model. Implement a homography estimation function with normalisation and with the RANSAC algorithm named `homography_w_normalisation_ransac`. Apply the function to feature point correspondences obtained from the previous question and list the 3×3 homography matrix H that you have calculated. (3 marks).

5.1 Code overview

```

def calculate_distances(h_mat, u2_trans, v2_trans, u_base, v_base):
    # Convert coordinates to homogeneous coordinates
    points_trans = np.column_stack((u2_trans, v2_trans, np.ones_like(u2_trans)))
    points_base = np.column_stack((u_base, v_base, np.ones_like(u_base)))
    # Warp points_trans using the estimated homography
    warped_points_trans = np.dot(h_mat, points_base.T).T
    warped_points_trans /= warped_points_trans[:, 2:]
    # Calculate Euclidean distances between warped_points_trans and points_base
    distances = np.sqrt(np.sum((warped_points_trans[:, :2] - points_trans[:, :2])**2, axis=1))
    return distances

def homography_w_normalisation_ransac(u_trans, v_trans, u_base, v_base, img_base, img_trans, dis_thresh, inlier_thresh, max_iter):
    best_h_mat = None
    best_inliers = []

    for _ in range(max_iter):
        # Randomly select a subset of correspondences
        indices = random.sample(range(len(u_trans)), k=4)

```

```

        selected_u_trans = [u_trans[i] for i in indices]
        selected_v_trans = [v_trans[i] for i in indices]
        selected_u_base = [u_base[i] for i in indices]
        selected_v_base = [v_base[i] for i in indices]

        # Estimate homography with normalization using the
        # selected correspondences
        h_mat = homography_w_normalisation(selected_u_trans
, selected_v_trans,
                                           selected_u_base,
        selected_v_base,
                                           img_base, img_tr
ans)

        # Calculate distances between estimated homography
        # and all correspondences
        distances = calculate_distances(h_mat, u_trans, v_t
rans, u_base, v_base)

        # Find inliers based on distance threshold
        inliers = np.where(distances < dis_thresh)[0]

        # Check if the number of inliers exceeds the curren
        t best

        if len(inliers) > len(best_inliers) and len(inliers
) > inlier_thresh:

            best_h_mat = h_mat
            best_inliers = inliers

    return best_h_mat

```

5.2 Code explanation

The operation method of RANSAC is as follows, (from slide)

RANSAC robust estimation: Repeat for N samples

- (a) Select 4 correspondences and compute H with DLT
- (b) Calculate the distance $d^{\wedge}$ for each putative match
- (c) Compute the number of inliers consistent with H ($d^{\wedge} < t$)

Choose H with most inliers, below are the details of the code method

5.2.1 homography_w_normalisation_ransac

- 1) The `homography_w_normalisation_ransac` function accepts a series of input parameters, including `u_trans`, `v_trans` (coordinates on the transformed image), `u_base`, `v_base` (coordinates on the reference image), `img_base`, `img_trans` (image data of the reference image and transformed image), `dis_thresh` (distance threshold, Used to determine whether it is an interior point), `inlier_thresh` (inlier threshold, used to determine the minimum number of interior points) and `max_iter` (RANSAC iteration number).

```

def homography_w_normalisation_ransac(u_trans, v_trans, u_base,
    v_base, img_base, img_trans, dis_thresh, inlier_thresh, max_it
er):

```

- 2) Initialize `best_h_mat` to store the best homography matrix, and initialize

best_inliers to store the index of the best inliers.

```
best_h_mat = None
best_inliers = []
```

- 3) Start RANSAC iterations, the number of iterations is controlled by max_iter.(In a sense, it produces a result that is reasonable with a certain probability, and more iterations increase this probability.)

```
for _ in range(max_iter):
```

- 4) In each iteration, a part of matching points is randomly selected as samples. Use the random.sample() method to randomly select 4 indices from the index ranges of u_trans and v_trans, and store the corresponding coordinates in selected_u_trans and selected_v_trans respectively. Similarly, the corresponding coordinates are selected from u_base and v_base and stored in selected_u_base and selected_v_base.

```
indices = random.sample(range(len(u_trans)), k=4)
selected_u_trans = [u_trans[i] for i in indices]
selected_v_trans = [v_trans[i] for i in indices]
selected_u_base = [u_base[i] for i in indices]
selected_v_base = [v_base[i] for i in indices]
```

- 5) Call the homography_w_normalisation()(detail in Q3) function to estimate the homography matrix h_mat with normalization using the selected matching points.

```
h_mat = homography_w_normalisation(selected_u_trans, selected_v_trans, selected_u_base, selected_v_base, img_base, img_trans)
```

- 6) Computes the distance between the estimated homography matrix and all matching points. Call the calculate_distances() function(will be explained in detail later), pass in h_mat, u_trans, v_trans, u_base and v_base, and calculate the distance list.

```
distances = calculate_distances(h_mat, u_trans, v_trans, u_base, v_base)
```

- 7) Based on the distance threshold dis_thresh, find the index of the inlier. Using the np.where() function, find the indices whose distance is less than the threshold, store them in inliers.

```
inliers = np.where(distances < dis_thresh)[0]
```

- 8) Check whether the number of current inliers exceeds the current optimal number of inliers and exceeds the inlier threshold inlier_thresh. If yes, update best_h_mat and best_inliers.

```
if len(inliers) > len(best_inliers) and len(inliers) > inlier_thresh:
    best_h_mat = h_mat
    best_inliers = inliers
```

- 9) After all iterations, return best_h_mat storing the best homography matrix.

5.2.2 Calculate_distances

- 1) Merge u2_trans and v2_trans, u2_base and v2_base on the transformed image

into points_trans and point_base in homogeneous coordinate form.

```
points_trans = np.column_stack((u2_trans, v2_trans, np.ones_like(u2_trans)))
points_base = np.column_stack((u_base, v_base, np.ones_like(u_base)))
```

- 2) Transform the points_base using the estimated homography h_mat to get the transformed coordinates warped_points_trans.

```
warped_points_trans = np.dot(h_mat, points_base.T).T
```

- 3) Normalizes warped_points_trans by dividing each coordinate point by its third component. The normalization operation is performed by dividing each element of the third column by itself by warped_points_trans[:, 2:] .

```
warped_points_trans /= warped_points_trans[:, 2:]
```

- 4) Computes the Euclidean distance between the transformed coordinate warped_points_trans and the original warped image coordinate points_trans . Use (warped_points_trans[:, :2] - points_trans[:, :2])**2 to calculate the square of the Euclidean distance between two points, and then sum all dimensions by np.sum() to get a shape of (N,) a one-dimensional array of . Finally, take the square root of np.sqrt() to get the final Euclidean distance array distances.

```
distances = np.sqrt(np.sum((warped_points_trans[:, :2] - points_trans[:, :2])**2, axis=1))
```

- 5) Returns a list of distances for all points

6. Warp the mountain1.jpg image according to the calculated homography matrix and stitch the warped image with the mountain2.jpg image for a more complete view of the mountain(2 marks).

6.1 Code implement

- 1) Convert coordinates from point form to u, v form, selected_points1 and selected_point2 from Q4

```
u_base = [point[0] for point in selected_points1]
v_base = [point[1] for point in selected_points1]
u_trans = [point[0] for point in selected_points2]
v_trans = [point[1] for point in selected_points2]
```

- 2) Call the function in Q5 to get the h matrix

```
h_mat = homography_w_normalisation_ransac(u_trans, v_trans, u_base, v_base, mountain1, mountain2, dis_thresh = 30, inlier_thresh = 50, max_iter = 10000)
```

The choice of dis_thresh = 30 suggests that any transformed point that is more than 50 units away from its.

The choice of dis_thresh = 50 suggests that any transformed point that is more than 50 units away from its.

Perform 10,000 iterations to find a more suitable homography.

- 3) Warp the first image (img1) according to the homography matrix (h_mat) using the cv2.warpPerspective() function. The resulting image is stored in img1_warped.

```
img1_warped = cv2.warpPerspective(img1, h_mat, (img1.shape[1] + img2.shape[1], img1.shape[0]))
```

- 4) Create a new image (panorama) with dimensions large enough to hold the warped image (img1_warped) and the second image (img2). The image is initialized with zeros and has a shape determined by the maximum height between img1_warped and img2, the width of img1_warped, and 3 channels for color.

```
panorama = np.zeros((max(img1_warped.shape[0], img2.shape[0]), img1_warped.shape[1], 3), dtype=np.uint8)
```

- 5) Copy the second image (img2) to the left side of the panorama image. This is done by specifying the appropriate indices to copy the pixels.

```
panorama[0:img2.shape[0], 0:img2.shape[1]] = img2
```

- 6) Create a binary mask by checking if all channels in the panorama image are zero. This mask will be used to determine where to copy the warped image.

```
mask = np.all(panorama == 0, axis=2)
```

- 7) Copy the pixels from the img1_warped image to the panorama image using the created mask. This effectively places the warped image onto the panorama in the correct location.

```
panorama[mask] = img1_warped[mask]
```

- 8) Display the stitched image using the plt.imshow() function from matplotlib. The plt.figure() function sets the size of the figure, plt.imshow() displays the image, plt.axis('off') removes the axis labels, and plt.show() shows the plot.

```
plt.figure(figsize=(10,10))
```

```
plt.imshow(panorama)
```

```
plt.axis('off')
```

```
plt.show()
```

6.2 Result visualization

Using the above parameters and functions, the following results can be obtained



6.3 Observation and analysis

When stitching images to create a panorama, it's normal to observe some imperfections like a visible seam line and a black border. The seam line occurs due to blending the images together. Different blending methods can result in a visible boundary or slight differences in brightness and color along the seam. The black border is a result of the warping process. In areas where there are no corresponding pixels, the panorama image is initialized with zeros, creating the black border.

7. **Theoretically or empirically discuss the factors that affect the rectified results, e.g., the distance between the corresponding points, e.g., the selected points and the warped ones. (2 marks).**

7.1 Implement Method and code

Here I will manually select points to observe the factors that affect the results of rectified

- 1) Use the matplotlib.pyplot.imshow function to display the mountain1.jpg image, and then use the matplotlib.pyplot.ginput function to manually select some corresponding points. Save the coordinates of these points in variables.

```
plt.imshow(img1)
points1 = plt.ginput(n=4, show_clicks=True)
plt.close()
```

- 2) Use the matplotlib.pyplot.imshow function to display the mountain2.jpg image, then use the matplotlib.pyplot.ginput function again to manually select the points corresponding to the points selected in the first step. Save the coordinates of these points in another variable.

```
plt.imshow(img2)
points2 = plt.ginput(n=4, show_clicks=True)
plt.close()
```

- 3) Use the saved corresponding point coordinates to call the homography_w_normalisation function to calculate the homography matrix.

```
u1 = [p[0] for p in points1]
v1 = [p[1] for p in points1]
u2 = [p[0] for p in points2]
v2 = [p[1] for p in points2]
# Calculate homography matrix
H = homography_w_normalisation(u2, v2, u1, v1, img1, img2)
```

- 4) Perspective transform (warp) the mountain1.jpg image using the computed homography matrix.

```
warped_img1 = cv2.warpPerspective(img1, H, (img1.shape[1], img1.s
hape[0]))
```

- 5) Stitch the perspective-transformed image with the mountain2.jpg image to form a complete perspective. Use the matplotlib.pyplot.imshow function to display the final stitched image.

```

panorama = np.zeros((max(img2.shape[0], warped_img1.shape[0]), im
g2.shape[1] + warped_img1.shape[1], 3), dtype=np.uint8)
panorama[0:img2.shape[0], 0:img2.shape[1]] = img2
panorama[0:warped_img1.shape[0], img2.shape[1]:] = warped_img1
plt.imshow(panorama)
plt.axis('off')
plt.show()

```

7.2 Observation

- 1) I randomly selected points on the mountains of the two pictures, and the result I got is this. Among them, the red point on mountain1 is the point I selected, the blue point on mountain2 is the point I selected, and then the third picture is the result of mountain1 wrapping according to the h matrix.



Figure 1.1 mountain1 with select points



Figure 2.1 mountain2 with select points

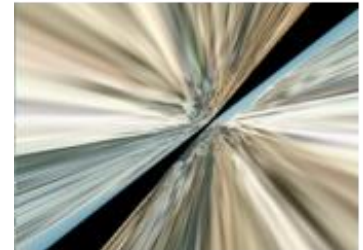


Figure 3.1 wrapping mountain1

According to the results, it can be seen that four points were randomly selected on the mountains of the two pictures, and the results obtained were very poor and severely distorted.

- 2) This time I randomly selected points in the sky on the two pictures, and the results are as follows



Figure 1.2 mountain1 with select points



Figure 2.2 mountain2 with select points



Figure 3.2 wrapping mountain1

t

It can be seen that the obtained image after wrapping hardly includes mountains, all of which are part of the sky, and there is almost no useful information.

- 3) Here, according to the mountains and trends, I choose regular points, such as the peaks on the same mountain in the two pictures, and the results are as follows,



Figure 1.3 mountain1 with select points



Figure 2.3 mountain2 with select points

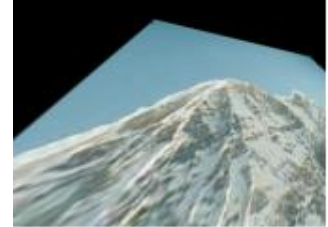


Figure 3.3 wrapping mountain1

According to the results, it can be seen that although the effect is not particularly perfect, compared with the previous results, the mountain after wrap has a certain shape. This imperfection stems from the fact that the human eye cannot quite pick out similar points.

In general, the third case performs best.

7.3 Analysis

From the above observations, it can be found that good results should be obtained. The selected point is preferably located on the subject of the image (in this case, the mountain). The selected point features should be similar (such as select the same peak). In conclusion, the factors that affect the rectified results as follow:

1. The selection of corresponding points is very important, because it directly affects the final correction result. Selecting accurate corresponding point pairs means selecting points with similar characteristics in the two images, which can ensure the accuracy and stability of the matching. If the selection of corresponding points is inaccurate or there is a mismatch, it will lead to distortion or inaccuracy of the correction result.

2. The distance between corresponding points is also an important factor. Larger distances may cause correction results to be affected by larger distortions. This is because large distances may introduce larger errors when computing the transformation matrix, and may cause inaccuracies in the mapping. On the contrary, smaller distances can provide more accurate rectification results because they represent matches between more similar feature points.

Therefore, when selecting corresponding points, we should try our best to choose those point pairs with small distance and obvious common features. This provides more stable and accurate correction results.